

Chapter-10

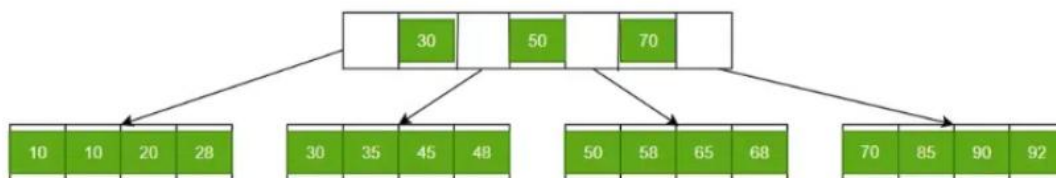
Definition of B+ Tree

A **B+ Tree** is a **self-balancing tree data structure** commonly used in **databases** and **file systems** for efficient indexing. It is an extension of the **B Tree**, where all values (records) are stored in the **leaf nodes**, and the internal nodes store only **keys** to guide searches. This structure is ideal for applications that require **sorted data** and support for **range queries**.

The key properties of a B+ Tree are:

- **Balanced:** All leaf nodes are at the same level, ensuring fast access.
 - **Sorted:** Keys in leaf nodes are stored in sorted order.
 - **Efficient for disk-based storage:** Internal nodes store only keys, optimizing disk I/O operations.
-

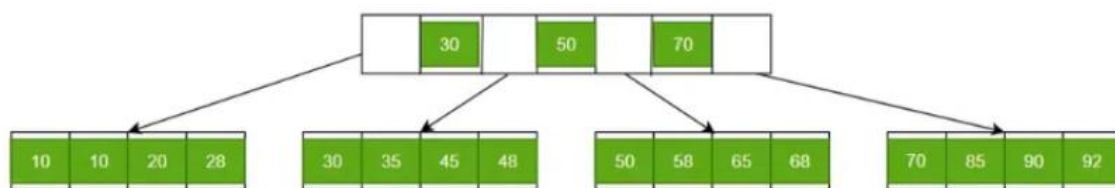
Structure of B+ Trees



- **Root Node:** The topmost node in the tree. It can be an internal node or a leaf node if the tree is small.
- **Internal Nodes:** These nodes store only **keys** (no data records). They **point to child nodes** and guide the search process. Internal nodes are used to traverse the tree.
- **Leaf Nodes:** These nodes contain **actual data records** or **pointers to data**. They are linked to each other in a **doubly linked list**, which is useful for efficient **range queries**.

- **Balanced Tree:** The B+ Tree is always **balanced**, meaning all leaf nodes are at the same level, ensuring **$O(\log n)$** time for search, insertion, and deletion operations.
- **Order of the Tree:** The order (m) defines the maximum number of children per internal node. Each internal node stores **$m-1$ keys** and **m children**, while each leaf node stores **$m-1$ keys** and **m data entries**.

Searching for a Record in a B+ Tree



Searching in B+ Tree

1. **Start at the Root Node:** Begin the search at the **root node**, which contains keys to guide the search.
2. **Traverse Internal Nodes:** Move through the internal nodes by comparing the search key with the keys in the current node and following the appropriate child pointer.
3. **Reach the Leaf Node:** Once a leaf node is reached, the search is confined to the **sorted keys** stored there.
4. **Return the Record:** If the key is found in the leaf node, return the corresponding data record. If not, return "record not found."

Insertion in a B+ Tree

1. **Find Correct Leaf Node:** Start by searching for the appropriate **leaf node** where the new key and record should be inserted.
2. **Insert the Key and Record:** Insert the new key and its associated record into the leaf node in sorted order.
3. **Handle Overflow:** If the leaf node overflows (contains more than **$m-1$ keys**), it splits:

- The node is divided into two, and the **median key** is pushed up to the **parent node**.
 - 4. **Update Parent Nodes:** If the parent node overflows, repeat the process: split the parent and push the median key up until the root.
 - 5. **Increase Tree Height:** If the **root node** overflows, a new root is created, increasing the height of the tree.
-

Deletion in a B+ Tree

1. **Search for the Key to Delete:** Find the **leaf node** containing the key to be deleted.
2. **Delete the Key and Record:** Remove the key and the associated record from the leaf node.
3. **Handle Underflow:** If the leaf node now has fewer than $\lceil m/2 \rceil$ keys, it may need to be **merged** with a sibling or **redistribute keys**:
 - The parent node is updated by removing the key that pointed to the merged node.
4. **Propagate Changes Upward:** If the parent node underflows, it is also merged or redistributed. This process continues up to the root.
5. **Update the Root:** If the root becomes empty (only one child left), the tree height decreases by one.

Advantages of B+ Trees in DBMS

1. **Efficient Searching ($O(\log n)$)**
 - Since B+ Trees are balanced, searches always take **logarithmic time**, making lookups fast even for large datasets.
 - Only the **leaf nodes** store actual data, making traversal faster in internal nodes (which store only keys).
2. **Efficient Range Queries**
 - **Leaf nodes are linked** in a **doubly linked list**, allowing for **fast sequential access** to records.

- Ideal for queries like **SELECT * FROM table WHERE key BETWEEN X AND Y**.

3. **Balanced Tree Structure**

- B+ Trees always remain **balanced**, meaning **all leaf nodes are at the same depth**.

4. **Faster Insertions and Deletions**

- Since B+ Trees remain balanced, insertions and deletions do not cause performance degradation.

5. **Disk Optimization (Reduced I/O Operations)**

- **Internal nodes contain only keys**, meaning **more keys fit in memory**, reducing disk accesses.
- Optimized for **database indexing**, as fewer disk reads/writes are needed.

6. **Supports Ordered Data & Sorting**

- Since keys in leaf nodes are **sorted**, B+ Trees are naturally efficient for **ORDER BY** queries.
- Useful for sorted retrievals and indexing.

7. **Scalability for Large Datasets**

- Can efficiently handle **millions of records** due to their **logarithmic height**.

8. **Multi-Level Indexing**

- B+ Trees are widely used in **database indexing**, ensuring quick retrievals using primary and secondary indexes.
 - Often used in **file systems** and **relational databases** for efficient searching.
-

Chapter-13

Field

A **field** is the smallest unit of data in a storage system. It represents a single data element.

- **Example:**
 - A "Student Name" field stores names like "John Doe".
 - A "Phone Number" field stores values like **+1234567890**.
 - **Characteristics:**
 - A field holds a single value.
 - Each field has a **data type** (e.g., integer, string, date).
 - Fields combine to form **records**.
-

2. Record

A **record** is a collection of related fields that represent a complete unit of information.

- **Example:** A student record may contain:

Student ID	Name	Age	Course
101	John Doe	22	CS
102	Alice	21	IT

- **Characteristics:**
 - A record groups multiple **fields** together.
 - Each record represents **one entity** (e.g., one student, one employee).
 - Records are stored in **files**.
-

3. File

A **file** is a collection of records stored together. It represents a set of data about multiple entities of the same type.

- **Example:** A **Student Database File** stores multiple student records.

Student ID	Name	Age	Course
------------	------	-----	--------

101	John Doe	22	CS
-----	----------	----	----

102	Alice	21	IT
-----	-------	----	----

103	Bob	23	EE
-----	-----	----	----

- **Characteristics:**
 - A file contains multiple **records**.
 - Files can be **structured** (databases) or **unstructured** (text files, logs).
 - A file can be stored on a **hard disk, SSD, or database system**.
-

File Organization

File organization refers to how records are stored and arranged in a file. It determines how efficiently data can be stored, accessed, and modified.

Types of File Organization:

Sequential:

- Records stored in order (e.g., by key).
- Best for sequential reads, slow for random access.

Indexed:

- Uses an index for fast access.
- Faster than sequential, but needs extra storage for the index.

Hashed:

- Uses a hash function to locate records.
- Fast for exact searches, not for range queries.

Clustered:

- Groups related records together.
- Reduces I/O, but harder to manage

Fixed-Length Records:

In a file or database, **fixed-length records** refer to records where the size (number of bytes) is the same for all entries. Each record in the file has a predefined, consistent length, regardless of the data stored in each field. This means that every field in a record will either contain its full value or have some padding (empty space) if the data is shorter than the defined length.

Characteristics of Fixed-Length Records:

1. **Uniform Size:** All records have the same length, making it easy to calculate the location of any record in the file.
2. **Padding:** If the data in a field is smaller than its allocated size, the remaining space is usually padded with empty characters (e.g., spaces or null values).
3. **Faster Access:** Since the length of each record is fixed, it's easier to compute the position of any record directly (e.g., using an offset). This allows for faster access to records.
4. **Simple Structure:** Fixed-length records are simple to implement and understand because the size of each record is constant.

Variable-Length Records

Variable-length records are records where the size of each record can vary, depending on the actual data stored in each field. Unlike fixed-length records, where every record has the same size, variable-length records allow each record to use only the amount of space it needs, which can save space but introduces more complexity in terms of access and storage management.

Characteristics of Variable-Length Records:

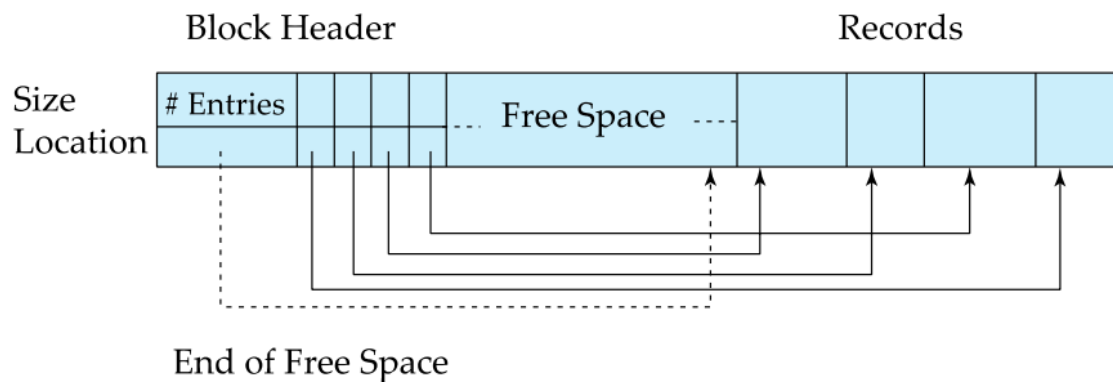
1. **Dynamic Size:** Each record's size depends on the actual content of the fields. The fields in the record are not pre-allocated a fixed number of bytes.
2. **No Padding:** Since each field can vary in length, there is no need for padding or unused space in the record.
3. **Complexity:** While more efficient in terms of storage, managing variable-length records is more complex because the system must track the size of each record and its individual fields.

Comparison between fixed-length and variable-length records:

Aspect	Fixed-Length Records	Variable-Length Records
Size	Predefined, same size for all records.	Size varies based on data.
Efficiency	Can waste space.	Space-efficient, no waste.
Access Speed	Faster due to fixed size.	Slower due to dynamic size.
Flexibility	Less flexible, same structure for all records.	More flexible, different sizes for each record.
Complexity	Simple to manage.	More complex due to varying sizes and offsets.
Space Wastage	May waste space for smaller data.	Minimal waste, uses only required space.
Examples	Used in systems with uniform record sizes.	Used in systems with varying data (e.g., documents).

Variable-Length Records: Slotted Page Structure

The **slotted page structure** is commonly used in database systems to store **variable-length records** efficiently. It is an approach that helps manage the varying sizes of records while keeping data organized and easily accessible.



Slotted Page Structure

This structure helps manage variable-length records efficiently. Here's how it works:

1. Block Header:

- This part contains important information about the records within the page.
- It holds:
 - **Number of Entries:** How many records are stored on the page.
 - **End of Free Space:** Indicates where the free space starts, which is the space available for new records.
 - **Location and Size of Each Record:** A list showing where each record is located within the page and how big it is.

2. Free Space:

- This is the area in the page where new records can be added.
- When records are deleted, their space can be reused to store new records, thus optimizing space usage.

3. Records: Stored at specific positions in the page, with their length and offset in the slot directory.

Sequential File Organization

Sequential file organization refers to storing records in a file in a specific, ordered sequence, typically based on one or more key fields. In this type of organization, records are stored one after the other.

File Operations: Deletion and Insertion

- **Deletion:**
 - Use **pointer chains**: When a record is deleted, it's not immediately removed. Instead, a pointer is used to mark the space as free, and the pointer helps to manage deleted records.
- **Insertion:**
 - Find the position where the new record should go.
 - If there's free space, insert the record there.
 - If no free space, place the record in an **overflow block** (extra space allocated for new records).
 - In both cases, update the pointer chain to track free spaces and overflow blocks.
- **Reorganization:**
 - From time to time, the file may need to be **reorganized** to keep records in order and remove fragmentation.

In short, **deletion** marks space as free using pointers, **insertion** finds space or uses overflow blocks, and **reorganization** helps maintain order.

Chapter-14

1. Basic Concepts

- **Indexing:** Indexing is a technique used to speed up data retrieval operations in a database. It improves query performance, particularly for search operations, by providing a more efficient way to access data than scanning the entire table.
- **Search Key:** An attribute or set of attributes used to locate records (e.g., Student ID).
- **Index File:** A separate file storing index entries, each containing:
 - Search Key
 - Pointer to the actual record in the main file.
- **Types of Indices:**
 - **Ordered Indices:** Search keys stored in sorted order for faster searching (e.g., binary search).
 - **Hash Indices:** Uses a hash function to distribute keys into "buckets" for constant time search operations ($O(1)$).

2. Index Evaluation Metrics: To measure the efficiency of an index, we use certain evaluation metrics. These metrics help determine how well an index performs for different operations

- **Access Time:** Time taken to locate a record using the index.
 - **Insertion Time:** Time to add new entries.
 - **Deletion Time:** Time to remove entries.
 - **Space Overhead:** Additional storage required for the index.
-

3. Ordered Indices

An index where entries are stored in a sorted order based on the search key, helping in fast data retrieval.

Types of Ordered Indices:

- **Clustering Index (Primary Index):** Specifies sequential order of the file; typically on the primary key.
 - **Secondary Index (Non-Clustering Index):** Specifies an order different from the sequential order of the file.
 - **Index-Sequential File:** Sequential file ordered on a search key with a clustering index on it.
-

4. Dense and Sparse Index Files

- **Dense Index:** Index record for every search-key value.
- **Sparse Index:** Index records only for some search-key values.
 - Uses less space but generally slower than dense indices.
 - Suitable for clustered indices with an index entry for every block in the file.

Secondary Index: An index that helps find data faster but does not follow the file's order. It stores a list of search keys with pointers to the actual records. It must have an entry for every record.

5. Multilevel Index

An index with multiple layers, where a higher-level index points to lower-level indexes instead of directly pointing to data. This reduces the size of each index and speeds up searching.

- Used when the index does not fit in memory.
- **Outer Index:** Sparse index of the basic index.
- **Inner Index:** The basic index file.
- Additional levels can be created if even the outer index is too large.

6. Indices on Multiple Keys

A search key made up of multiple fields, combined together to uniquely identify a record. It's used when a single field isn't enough to uniquely search for a record.

- **Composite Search Key:** Index on multiple attributes, sorted lexicographically (e.g., (name, ID)).
- Enables querying on individual or combined attributes efficiently. Values are sorted lexicographically

E.g. (John, 12121) < (John, 13514) and (John, 13514) < (Peter, 11223)

7. B+-Tree Index Files

- **Structure:**
 - Balanced tree structure with all paths from root to leaf of equal length.
 - **Leaf Nodes:** Store search-key values and pointers to actual records.
 - **Non-Leaf Nodes:** Form a multi-level sparse index on leaf nodes.
 - **Properties:**
 - Efficient search operations with logarithmic complexity.
 - Handles insertions and deletions efficiently by restructuring the index.
 - **Operations:**
 - **Insertion:** If a leaf node is full, it is split and updates propagate upwards.
 - **Deletion:** Nodes under minimum occupancy are merged or redistributed.
-

8. B+-Tree File Organization

- **Leaf Nodes Store Records:** Records are clustered, aiding efficient range queries.
- **Space Utilization:** To maximize space, nodes redistribute pointers or involve multiple siblings during splits/merges.

Chapter-18

Concurrency control ensures that multiple transactions can run simultaneously without causing inconsistencies in the database.

Lock-Based Protocols

A **lock-based protocol** is a mechanism used in database systems to manage concurrent access to data by multiple transactions, ensuring data consistency and preventing conflicts

Types of Locks:

- **Shared (S) Lock:** Allows reading but no writing.
- **Exclusive (X) Lock:** Allows both reading and writing.

Lock Compatibility Matrix:

- Multiple transactions can hold an **S-lock** on the same data.
- If a transaction holds an **X-lock**, no other transaction can hold any lock on that data.

	S	X
S	true	false
X	false	false

Example:

- Transaction T1 reads a record with an S-lock.

- Transaction T2 requests an X-lock on the same record → T2 must wait.

Deadlock

Deadlock occurs when two or more transactions are waiting for each other to release locks, preventing any of them from proceeding.

T_3	T_4
lock-X(B) read(B) $B := B - 50$ write(B) lock-X(A)	 lock-S(A) read(A) lock-S(B)

Example Scenario:

- Transactions:**
 - T3:** Wants to lock resource B and is currently holding a lock on A.
 - T4:** Wants to lock resource A and is currently holding a lock on B.

Locking protocols are used in database systems to manage access to data and ensure concurrency control.

Two-Phase Locking Protocol (2PL)

The **Two-Phase Locking Protocol (2PL)** is a concurrency control mechanism used to ensure serializability in database systems. It works by requiring each transaction to follow two phases:

- Growing Phase:** In this phase, the transaction can acquire locks (shared or exclusive) on data items but cannot release any locks.

2. **Shrinking Phase:** Once a transaction starts releasing locks, it enters the shrinking phase, where no new locks can be acquired, and it can only release existing ones.

Lock Conversions

In a **Two-Phase Locking Protocol with Lock Conversions**, transactions can upgrade or downgrade locks during the growing and shrinking phases:

- **Growing Phase:**
 - Acquire a **shared lock (S)** on an item.
 - Acquire an **exclusive lock (X)** on an item.
 - **Upgrade** from a shared lock (S) to an exclusive lock (X).
- **Shrinking Phase:**
 - Release a **shared lock (S)**.
 - Release an **exclusive lock (X)**.
 - **Downgrade** from an exclusive lock (X) to a shared lock (S).

Deadlock handling refers to techniques used in database systems to detect, prevent, or recover from deadlocks, where transactions wait indefinitely for resources held by each other. There are three main strategies for handling deadlocks:

1. Deadlock Prevention:

- This approach prevents deadlocks by ensuring that the system never enters a state where deadlock could occur. It is done by imposing constraints on how transactions can acquire locks.

Methods:

- **Hold and Wait Prevention:** Transactions must acquire all the locks they need at once before proceeding. This avoids waiting for other resources while holding locks.

- **No Preemption:** If a transaction holds a lock and requests another, the system may preemptively release the lock(s) held by the transaction to avoid deadlock.
- **Circular Wait Prevention:** Transactions must acquire locks in a predefined order to prevent cycles from forming (e.g., always acquire lock on A before lock on B).

2. Deadlock Detection:

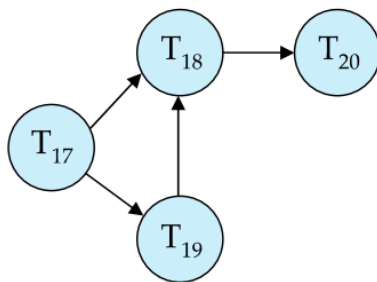
- In this approach, the system allows deadlocks to occur but periodically checks for cycles in the wait-for graph to detect deadlocks.

Methods:

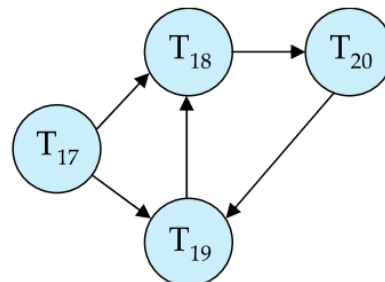
- **Wait-for Graph:** Transactions are represented as nodes, and directed edges represent waiting relationships (i.e., one transaction is waiting for another to release a lock). A cycle in this graph indicates a deadlock.
- Once a deadlock is detected, the system typically chooses one or more transactions to **abort** and releases their locks, allowing other transactions to proceed.

Example:

- Transaction T1 waits for T2 to release a lock, T2 waits for T3, and T3 waits for T1. This forms a cycle in the wait-for graph, indicating a deadlock.



Wait-for graph without a cycle



Wait-for graph with a cycle

ARIES Recovery Algorithm

1. Analysis Phase

- 1.1. Read log forward from the last checkpoint.
- 1.2. Identify active transactions and dirty pages in the buffer pool.

2. Redo Phase

- 2.1. Start from the earliest required LSN.
- 2.2. Redo all committed operations to ensure database consistency.
- 2.3. Skip already applied changes.

3. Undo Phase

- 3.1. Process uncommitted transactions in reverse order.
- 3.2. Rollback changes using log entries.
- 3.3. Write Compensation Log Records (CLRs) for undone operations.
- 3.4. Continue until all uncommitted transactions are undone.

◇ Ensures durability and consistency using **Write-Ahead Logging (WAL)**.

ARIES Recovery Phases

Analysis Phase

Scans the log forward from the last checkpoint.
Identifies active transactions and dirty pages.

Redo Phase

Scans the log forward again, applying all committed and in-progress updates to ensure database consistency.

Undo Phase

Scans the log backward, rolling back uncommitted transactions using the UNDO log records.